
Duma SDV: A Browser-Based Digital Twin of a Software-Defined Electric Vehicle

FNM

Software-Defined Electric Vehicle Design Challenge

Abstract

A software-defined vehicle (SDV) is a car whose behaviour is decided, and can be changed, by software. Its design is hard to judge from diagrams alone, because the interesting part is how the software modules coordinate. We present Duma SDV, a digital twin of a fictional rear-drive electric sedan that runs in a web browser, online or fully offline. Six electronic control units (ECUs) are separate software modules that exchange only messages on a simulated CAN bus: a vehicle control unit, battery management, motor control, an on-board charger, an instrument cluster and a telematics unit. They sit on physical plant models of the vehicle, battery, high-voltage circuit and thermal masses, advanced in deterministic 10 ms steps. A judge can power the car on, drive it, charge it on AC or DC, inject faults and install an over-the-air (OTA) update, while a 3D model, a driver dashboard and a live bus trace show every step. Automated tests hold the physics to the published figures of a production reference car: 0–100 km/h in 5.97 s (target 5.9 s), 498 km at a steady 100 km/h (510 km) and 34.0 min for a 10–80 % DC charge (37 min), all within the $\pm 10\%$ goal. Under a cell over-temperature fault the battery management unit halves its discharge allowance within 60 ms and the drive power follows it. The OTA update downloads, verifies and installs into an inactive firmware bank, restarts the car and unlocks a drive mode in 14.8 s. An optional co-pilot built on the Gemini API lets the driver talk to the car in English or Kiswahili, speaks up on its own when a trigger fires, and writes the summary of an exportable PDF vehicle report. It acts only through the commands the touchscreen sends, so the car's own interlocks decide. Every figure in this paper is generated from the simulator.

1 Introduction

An electric vehicle is increasingly a software product. Its supervisor decides how pedal travel becomes torque, when the high-voltage (HV) battery may connect, how braking is shared between the motor and the friction brakes, and how the car responds when a sensor reports a fault [1]. In a software-defined vehicle these decisions live in updatable software on a network of electronic control units (ECUs), and an over-the-air update can change the car after it is sold [12].

A design challenge for such a vehicle asks for its software ecosystem without a physical car. The usual answer is an architecture diagram and a mock-up screen. Judges then have to take on trust that the modules would work together: they cannot watch a fault spread from a sensor to a warning light, or see a software update change how the car drives.

Duma SDV makes the design something a judge can run and check. It is a digital twin [5] in the

practical sense: a model of the vehicle and of its software that behaves like the real system under the same inputs. Our contributions are:

- An ECU architecture in which every module is separate code that knows other modules **only through CAN messages** declared once in a catalogue. The architecture view and the tables in this paper are drawn from that catalogue, so the diagram cannot disagree with the code.
- A **deterministic, framework-free simulation core** that runs the same code headlessly in tests and behind a 60 fps browser view.
- Physics calibrated to a published reference car and **held within $\pm 10\%$** by automated tests.
- Complete software flows for **startup, driving, regenerative braking, AC and DC charging, fault diagnosis and OTA update**, each playable from a one-button guided demo and each verified by an end-to-end browser test.
- A **voice co-pilot that cannot override the car**: a large language model that reaches the vehicle only through a fixed whitelist of the touchscreen’s own commands, plus deterministic proactive warnings and a PDF service report (Section 5).

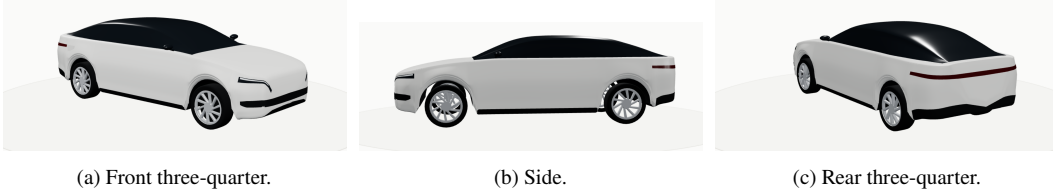


Figure 1: **The Duma SDV car on the app’s 3D stage.** A smooth, stylised fastback built in code at the reference car’s size, with no branding. These are screen captures of the running app (three.js), not offline renders.

2 Requirements and scope

The competition names five required scenarios; we add the OTA update as a sixth (Table 1). Each must run unaided in under a minute from the first click, from a public link and from a laptop with no network. The ranked trade-offs that decide conflicts are, in order: correctness of the physics and engineering, demo reliability, user experience, performance, code simplicity and delivery speed.

Table 1: **Scenarios and where they are verified.** Every scenario is scripted in the guided demo and has its own browser test that plays it from the demo.

Scenario	What the judge sees	Key ECUs
Startup	12 V wake, self-checks, pre-charge, contactors, READY; boot frames on the bus	VCU, BMS, MCU
Driving	Pedal to torque request, speed, power, SOC and range on the dashboard	VCU, MCU, IC
Regen	Negative power on lift-off, blended braking, energy recovered	VCU, MCU, BMS
Charging	AC through the on-board charger, DC around it; charge curve and time to target	VCU, BMS, OBC
Fault	DTC on the bus, derate or limp mode, plain-language warning, high-lighted module	BMS, MCU, VCU, IC
OTA update	Check, download, verify, install, restart; a new drive mode appears	TCU, VCU

Out of scope are real hardware, autonomous driving, any backend, and an open road network: the car drives on a rolling-road stage. The one cloud service is the Gemini API [3], called from the browser for the co-pilot and the report summary; everything else, including the report itself, runs with no network, and the AI controls say why they are unavailable when there is no key or connection. The vehicle is fictional. It takes its specifications from a production mid-size rear-drive sedan with a lithium iron phosphate (LFP) pack, which we call the *reference car* and never name.

3 System architecture

3.1 Layers

The application has three layers (Figure 2). The *simulation core* is plain TypeScript with no user-interface, 3D or browser dependencies; a lint rule rejects imports of React or three.js, the wall clock and unseeded random numbers. The *stores* hold the current simulation and the user-interface state. The *views* and the 3D stage only read snapshots and send inputs. A frame loop converts wall time into a whole number of simulation steps, scaled by a user-selectable time scale of $1\times$ to $120\times$, so a 35-minute charge can be watched in seconds. Beside the stores sits an optional *AI layer* (Section 5); it reaches the car only through the same commands the touchscreen sends.

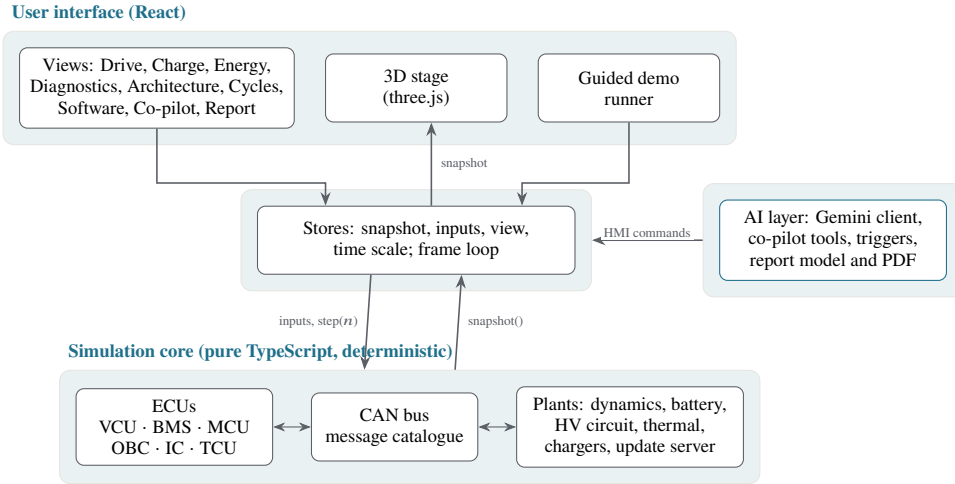


Figure 2: **Layers.** The simulation core has no user-interface or browser dependencies. The views, the 3D stage, the guided demo and the AI layer drive it only through inputs and read it only through snapshots; the core never imports the AI layer (a lint rule enforces both directions).

3.2 ECUs and the bus

The core separates two kinds of model. *Plants* are the physical world: longitudinal vehicle dynamics, the pack’s open-circuit voltage and internal resistance, the HV circuit with its contactors and DC-link capacitor, lumped thermal masses with two coolant loops [10], the AC wallbox, the DC charger and the update server. *ECUs* are software. An ECU senses its own plant signals (the BMS its cell temperatures, the MCU its DC-link voltage and rotor speed) and learns everything else from bus messages. No ECU reads another ECU’s state directly. This rule is what makes the architecture view truthful: if a signal matters to an ECU, it appears on the bus trace.

Figure 3 shows the six ECUs. The **VCU** is the supervisor: it senses the power button, pedals and gear selector, owns the switched 12 V supply of the other ECUs, runs the power state machine and the startup sequence, applies the gear interlocks, turns pedal travel into a torque request, decides derates, and estimates range. The **BMS** estimates state of charge (SOC), monitors cell temperature, controls the contactors and pre-charge, and publishes the charge and discharge limits. The **MCU** turns torque requests into motor torque within the motor envelope, and only while the VCU commands READY, the BMS reports closed contactors and the DC link it measures is charged. The **OBC** converts AC for the pack when the VCU and BMS both authorise it. The **IC** builds the dashboard, including its warnings, only from frames it receives, and marks a value unavailable when its frame goes stale. The **TCU** is the car’s link to the update server and runs the OTA client.

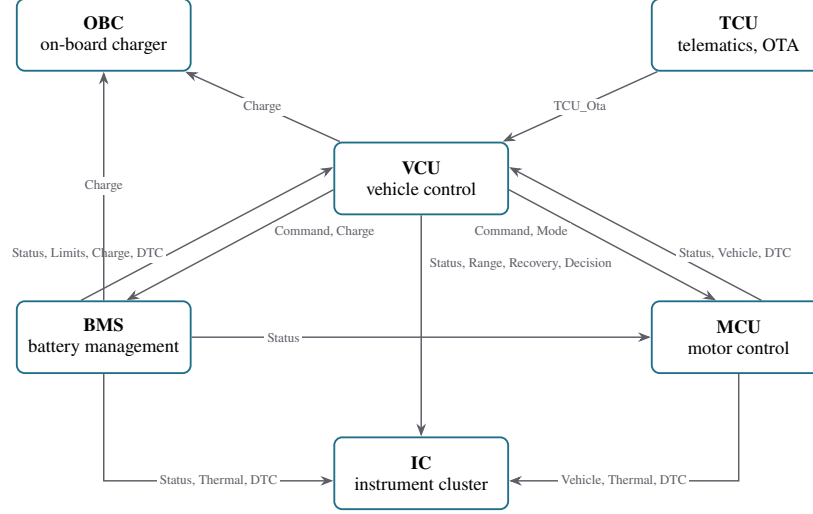


Figure 3: **ECUs and their main messages.** Arrows run from sender to subscriber; message names omit the sender prefix (BMS_Status appears as Status beside the BMS). The TCU also subscribes to boot, status, charge, speed and SOC frames to check the install preconditions. The OBC only listens. The full catalogue is Table 2.

3.3 Message catalogue

Every message is declared once, with its identifier, sender, period and signals with units and scaling (Table 2). The bus delivers frames sent in one step at the start of the next, so every hop costs at least one step, and a frame older than two or three periods counts as lost. The catalogue has 23 messages (18 periodic, from 10 ms to 1 s, and 5 event-driven boot frames that carry each ECU’s self-check result and software version) with 61 signals among 6 nodes. While driving the bus carries 500 frames per second. Identifiers group by sender: **0x1xx** VCU, **0x2xx** BMS, **0x3xx** MCU, **0x4xx** TCU. The bus models message timing and loss, not bit timing or arbitration [7]. The Architecture view (Figure 4) draws the same topology live, marks which ECUs are transmitting, and lists every frame in a trace that can be filtered by ECU or message and paused.

Table 2: **Message catalogue**, generated from the code. Period in ms; subscribers from the live topology.

ID	Message	Period	Signals	Subscribers
0x0F1	VCU_Boot	event	2	TCU
0x0F2	BMS_Boot	event	2	VCU
0x0F3	MCU_Boot	event	2	VCU
0x0F4	IC_Boot	event	2	VCU
0x0F5	TCU_Boot	event	2	–
0x100	VCU_Command	10	3	BMS, MCU
0x101	VCU_Status	100	5	IC, TCU
0x102	VCU_Range	1000	3	IC
0x103	VCU_Recovery	100	1	IC
0x104	VCU_Charge	10	7	BMS, IC, OBC, TCU
0x105	VCU_DTC	100	2	IC
0x106	VCU_DriveDecision	100	3	IC
0x107	VCU_Mode	100	1	MCU
0x200	BMS_Status	100	5	IC, MCU, TCU, VCU
0x201	BMS_Limits	100	2	IC, VCU
0x202	BMS_Charge	10	3	IC, OBC, VCU
0x203	BMS_DTC	100	2	IC, VCU
0x204	BMS_Thermal	100	1	IC
0x300	MCU_Status	10	4	BMS, VCU
0x301	MCU_Vehicle	20	1	IC, TCU, VCU
0x302	MCU_DTC	100	2	IC, VCU
0x303	MCU_Thermal	100	2	IC
0x400	TCU_Ota	100	4	VCU

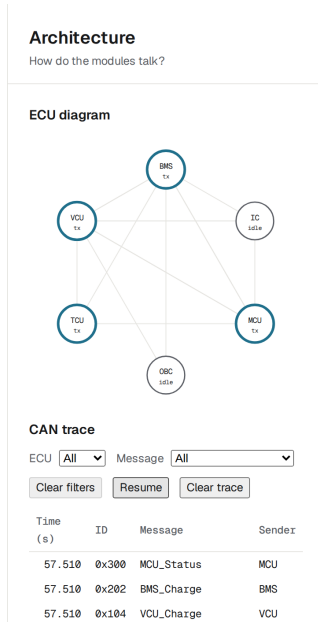


Figure 4: **The Architecture view while driving**, paused. The ECU diagram is built from the live bus topology; a node reads *tx* while it transmits and *idle* otherwise (the OBC only listens, the IC's frames are all received). Below it, the CAN trace lists frames with sim time, identifier, name and sender; selecting one shows its decoded signals.

4 Software design

4.1 A deterministic core

The simulation advances in fixed 10 ms steps (100 Hz). SI units are used throughout the core, and km/h, kW and kWh appear only at the user-interface boundary. The same initial state and input sequence always give identical snapshots and bus traces; tests check this for startup, driving, charging and the fault flow. Determinism lets a test drive the car through a 30-hour range run or a 35-minute charge in seconds, and it lets the guided demo and the browser tests replay exactly what a judge would see.

4.2 Power states and startup

The VCU runs the power state machine OFF → ACCESSORY → STARTING → READY, with CHARGING while an authorised session runs. Startup has five steps (Figure 5). On *wake* the VCU switches on the 12 V supply of the other ECUs and waits for their boot frames. In the *self-check* it requires three consecutive plausible BMS_Status frames with open contactors, fresh limit and fault frames, and the MCU in standby. *Pre-charge* closes the negative and pre-charge contactors so the DC-link capacitance (1.0 mF) charges through 200 Ω; the BMS reports done at 95 % of pack voltage. The main contactor then closes, and *READY* follows once the MCU confirms that the DC link it measures is within 10 % of pack voltage. Each step has a timeout, and a failed step powers the car down with a reason. The car reaches READY 1.62 s after Power on.

4.3 Torque path and drive modes

The VCU turns the accelerator into a torque request: pedal travel raised to a mode exponent, times the torque the motor can make at its current speed. It then applies three limits. The BMS discharge limit bounds *battery* power, so the VCU solves for the largest torque whose shaft power plus modelled motor and inverter loss plus the auxiliary load stays within it. The VCU's own derate caps shaft power and speed when a fault calls for it. A limiter fades torque out over the last 2 km/h below the speed limit. The MCU clamps the request to the motor envelope once more. Three drive modes change the maps (Table 3); a mode change blends the old map into the new over 0.5 s so the torque never steps.

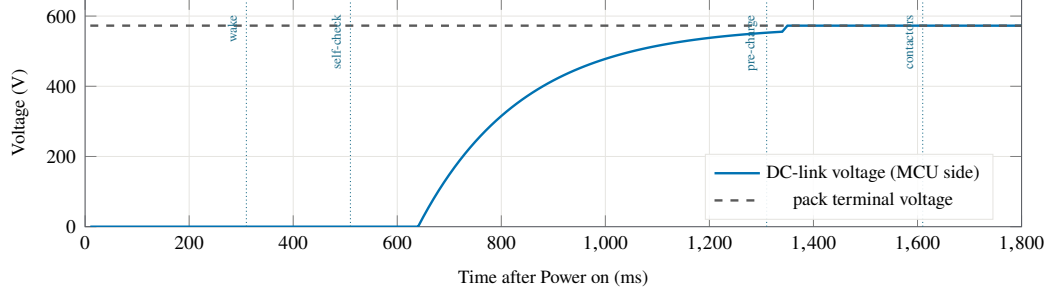


Figure 5: **Startup on the HV circuit.** Dotted lines mark the end of each step. The DC link charges through the pre-charge resistor, the main contactor closes onto a charged capacitor, and READY follows 1.62 s after Power on.

Table 3: **Drive modes.** Values other than the motor limits are our estimates; Sport arrives with VCU software 1.1.0 (Section 4.7). Energy on the WLTC phases [11], measured at the pack terminals.

	Eco	Normal	Sport
Pedal exponent	1.6	1.0	0.8
Shaft power cap (kW)	140	230	230
Lift-off regen deceleration (g)	0.20	0.15	0.15
Urban (Low phase, Wh/km)	146.4	144.8	144.8
Highway (Extra High phase, Wh/km)	183.3	183.2	183.2

4.4 Regenerative braking

On lift-off the VCU requests generator torque for 0.15 g in Normal, faded in between 0.5 and 5 m/s so the car does not creep backwards. The brake pedal can ask for up to 0.30 g of electric braking. Friction brakes fill whatever the pedal demands beyond the torque the MCU actually produces, so the driver’s deceleration never depends on the battery’s willingness to accept charge. Regen is refused when the BMS reports no charge allowance or a fault disables it, and the friction brakes then take the whole demand.

4.5 Charging

Plugging in requires P and a stopped car. The VCU and the BMS each authorise a session on their own charge messages, and the charger delivers current only while both frames are fresh and positive. AC passes through the on-board charger (11 kW); DC goes around it to the pack, following a taper from the charger’s 150 kW peak and the BMS allowance. The protocol with the charger itself [6] is outside the model.

4.6 Faults and safe states

Four faults can be injected (Table 4). The owning ECU raises a diagnostic trouble code (DTC) with an active or stored status in a bit field on its DTC message [9]. The VCU combines the DTC frames into one drive decision, which the IC turns into a plain-language warning with an icon and the affected module; the 3D car highlights that module. Clearing the stored records asks for confirmation (Figure 6). The responses follow the usual pattern of reducing capability to reach a safe state rather than stopping outright [8], but the model is not a safety analysis.

4.7 Over-the-air update

The OTA flow follows the structure of a managed software update [12]. The TCU checks the server, downloads a 48 MB VCU image at 8 MB/s (driving is allowed meanwhile), and verifies its signature and digest. It installs only when the driver confirms and the car is READY, in P, stopped, not charging and above 20 % SOC, all read from bus frames. The VCU holds two firmware banks. The image

Table 4: **Fault catalogue and the car's response.**

Fault	DTC	Owner	Warning	Response
Cell over-temperature	P0A7E	BMS	amber	Discharge allowance halved, charging and regen refused
Insulation fault	P0AA6	BMS	red	No drive torque, startup and charging refused
Motor over-temperature	P0A2F	MCU	amber	Limp mode: 30 kW, 50 km/h, no regen
12 V low	P0562	VCU	amber	Limp mode: 20 kW, 40 km/h; startup refused

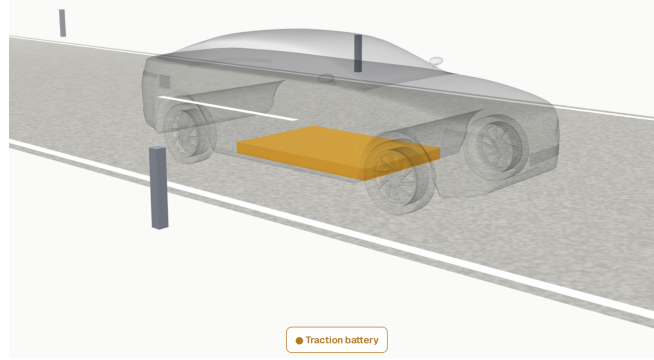


Figure 6: **X-ray view during a cell over-temperature fault while driving.** The body turns translucent and the faulted module, the traction battery in the floor, glows in the warning colour, with its name in words so the status never relies on colour alone.

is written to the inactive bank while the running firmware keeps working, and the VCU refuses to leave P until it is done. The VCU then powers the car down through its normal path, switches banks while asleep and powers back up. Its boot frame now reports version 1.1.0, and the TCU confirms the install only from that frame. Losing power during the install abandons the inactive bank and the car keeps its old version. Table 5 gives the timeline. Version 1.1.0 unlocks Sport mode, so the change is visible on the dashboard and in the drive.

Table 5: **OTA timeline**, measured in the simulator from Check for updates (install confirmed as soon as it is offered).

Event	Time (s)
Check complete, download starts	1.0
Download complete, verification starts	7.0
Ready to install; install starts	8.0
Inactive bank written; TCU asks for the restart	13.0
Car powered down, banks switched	13.1
VCU boot frame reports 1.1.0; TCU reports installed	13.4
Car READY again with Sport available	14.8

4.8 User interface and guided demo

Nine views each answer one question: what the car is doing (Drive), how full it is (Charge), where the energy goes (Energy), whether anything is wrong (Diagnostics), how the modules talk (Architecture), how efficient it is over a standard trip (Cycles), what software it runs (Software), what the car has to say (Co-pilot) and what a service engineer would need to know (Report). A one-page design contract fixes the type scale, spacing, colours and action verbs, and colour is used only for data and status, never alone. The 3D car is built in code, has named parts for fault highlighting and energy flow, and stays within 30 000 triangles.

Figure 7 shows five of the views and the guided demo.

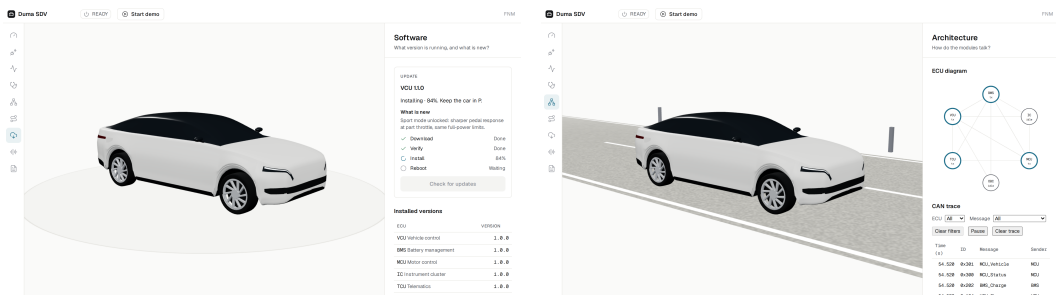
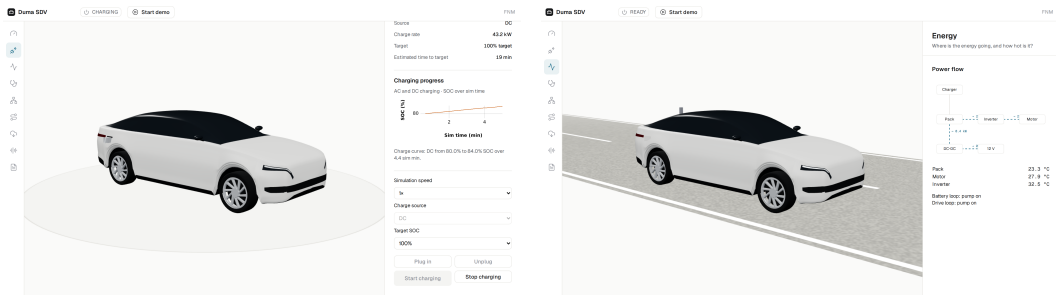
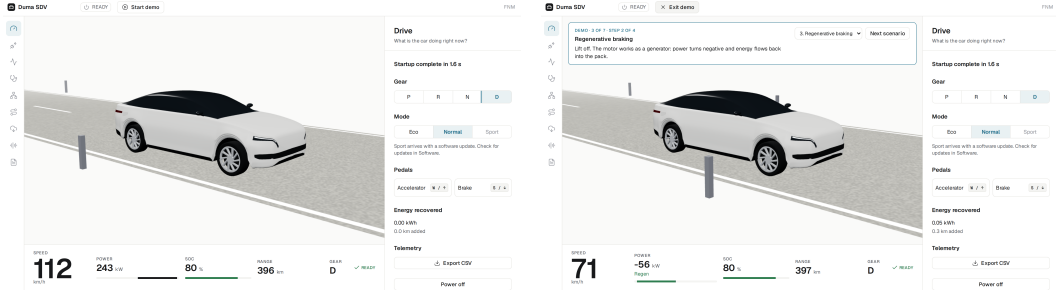


Figure 7: **The application at 1366 $\times$ 768**, captured from the production build. A left rail switches views, the 3D stage fills the middle, and the right panel answers the view's one question.

The guided demo plays all six scenarios from one button. Each scenario is a script of steps with a caption, a view, a time scale, the inputs to apply and the condition that ends the step. The demo drives the car only through the same inputs a driver uses. Each scenario starts from a fresh simulation prepared headlessly, so any scenario can be played on its own, and a step that does not finish within its timeout stops the demo with an error instead of hanging. The production build is a progressive web app whose service worker caches the whole application, so it runs at a venue with no network.

5 AI co-pilot and vehicle report

An SDV is also the platform for software that talks to the driver. We added three features on the Gemini API [3]: a voice co-pilot, proactive warnings, and a PDF report with an AI summary. The design question is the same as for any other module: what may it do to the car, and through which interface?

5.1 Integration and model choice

All AI calls go through one client interface with two operations: a text request that can demand JSON matching a schema, and a live voice session. A browser implementation wraps Google’s SDK; a scripted fake stands in for it in every test, so no test needs a key or a network. The owner sets the key and two models, a text model and a live voice model, in a local, git-ignored environment file, so switching models is an edit and a restart, not a code change. The key is bundled into local builds only, and a build that contains one is never published. Without a key or a connection the car works exactly as before.

5.2 A touchscreen with a voice

In a production SDV a voice assistant runs on the infotainment unit and sends the same requests as the touchscreen. The co-pilot follows that pattern. It streams microphone audio to the Gemini Live API [4] and plays the spoken reply, in English or Kiswahili. The model may call only 7 tools: 2 read the vehicle status and the fault records, and the rest set the drive mode, set the charge target, start or stop charging, and check for updates. Each act tool goes through an *HMI port* that calls exactly the store actions the touchscreen calls, advances the car one step, and reads the result back from the snapshot. The VCU, the charge path and the TCU therefore apply their own interlocks, and a refusal comes back with the car’s reason for the model to say. No tool presses a pedal, changes gear, powers the car, plugs a cable, clears a fault or installs an update; a test checks the whitelist. Table 6 shows tool calls answered by the real car. We considered a separate infotainment ECU that forwards requests over the bus; it would add catalogue messages and duplicate arbitration the VCU already does for touchscreen input, so the co-pilot stays outside the bus.

Table 6: **Co-pilot tool calls answered by the car**, generated by running each tool against the simulator. The message is what the model is given to say.

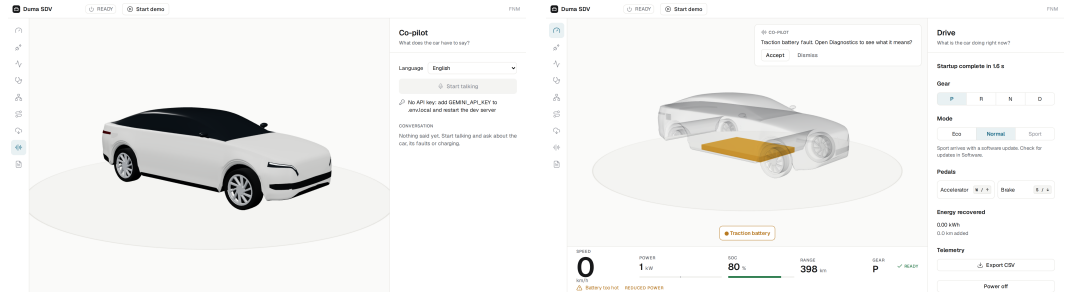
Driver asks	Tool	Outcome	Message
Switch to Eco	set_drive_mode	done	Drive mode set to Eco.
Switch to Sport before the update	set_drive_mode	refused: modeUnavailable	Sport needs the software update. Check for updates and install it from the Software screen.
Set the charge target to 85 percent	set_charge_target	refused: invalidArgument	Charge target must be one of 50, 60, 70, 80, 90, 100 percent.
Start charging, no cable plugged in	start_charging	refused: notPlugged	The cable is not plugged in. Someone has to plug it in first.
Stop charging with no session running	stop_charging	refused: notCharging	There is no charging session to stop.
Check for updates while driving	check_for_updates	done	Checking for updates.
Press the accelerator	press_accelerator	refused: unknownTool	There is no tool called press_accelerator.

5.3 Proactive warnings

The co-pilot also speaks up on its own, but rules decide when. Five triggers observe consecutive snapshots in sim time: a new DTC, the drive restriction starting, the pack reaching 45 °C, SOC falling to 20 % and to 10 %, and a finished charge. Each has a cooldown (120 s for a derate, 300 s for a hot pack), so the triggers are deterministic and tested like the rest of the core. One suggestion shows at a time, by priority. The language model only phrases it: a card on the stage shows a fixed English or Kiswahili sentence at once, which the text model may reword within 3 s, and an open voice session says it aloud. Nothing happens until the driver presses Accept or says yes, and Accept goes through the same HMI port. Table 7 lists what the sample run of Section 5.4 raised; Figure 8 shows the card.

Table 7: **Suggestions raised in the sample run**, times after the cell over-temperature fault is injected. The derate suggestion waits behind the fault suggestion.

Time (s)	Trigger	Words on the card	Action on Accept
0.01	newFault	Traction battery fault. Open Diagnostics to see what it means?	open diagnostics
0.18	derate	Power is limited: reduced power. Switch to Eco to go easy on the car?	switch to Eco

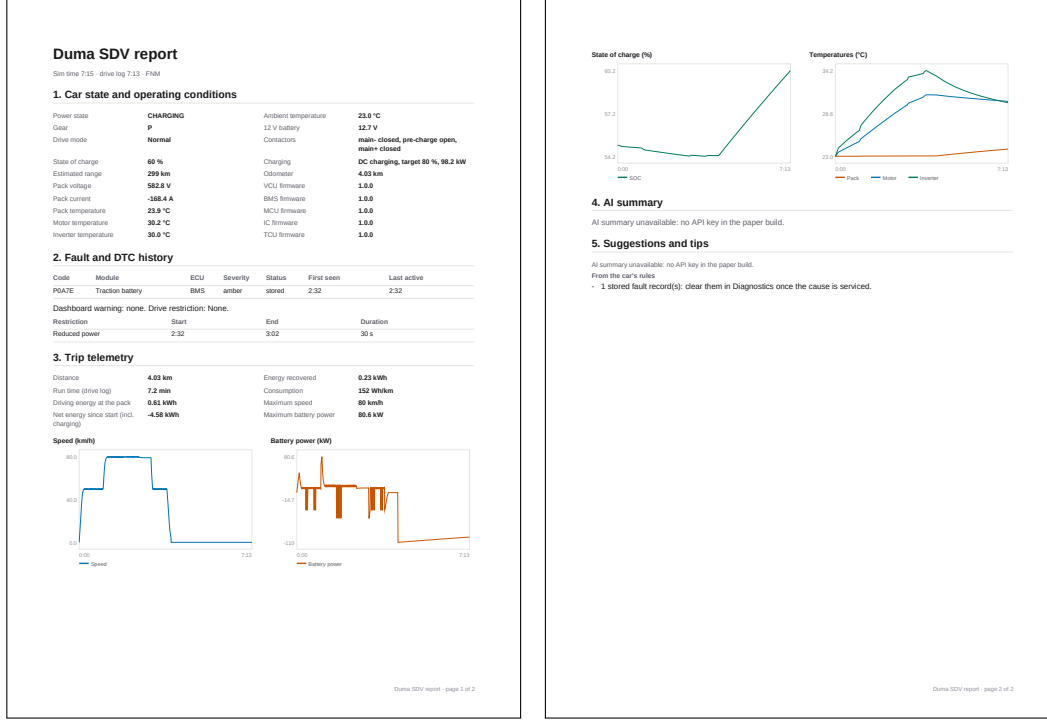


(a) Co-pilot view without a key: language, Start talking, and why it is unavailable. (b) A proactive suggestion after a cell over-temperature fault, waiting for Accept.

Figure 8: **The co-pilot in the app**, captured from the production build at 1366×768.

5.4 Vehicle report

The Report view exports a PDF in five sections: car state and operating conditions, fault and DTC history with each reduced-power or limp episode, trip telemetry with charts of speed, battery power, SOC and temperatures, an AI summary, and suggestions and tips. A pure function builds every figure and table from the snapshot, the drive log and the tracked episodes; the renderer only draws. Consumption counts energy only while a drive gear is engaged, so a charge in the same run cannot offset it. The text model receives the report data as compact JSON and must return a summary and up to five tips matching a schema, within 10 s; otherwise the section says why, and rule-based tips (active faults, a hot pack, low charge, high consumption, stored faults) always appear. Figure 9 is the report of a 7.2-minute sample run: 4.03 km at up to 80 km/h with the fault from Table 7, then a DC charge, built without a key.



(a) Page 1: state, faults and episode, trip figures and charts. (b) Page 2: charts, and the AI sections saying why they are empty.

Figure 9: **The vehicle report** of the sample run, generated by the simulator with no API key.

6 Vehicle model and calibration

The longitudinal model integrates tractive force against aerodynamic drag, rolling resistance and grade, with a rotational mass factor [2]. Motor and inverter loss is a four-term map in torque and speed. The pack is 172 LFP cells in series (550 V nominal, 82.5 kWh usable) with an SOC-dependent open-circuit voltage and a series resistance. Published figures of the reference car set the official parameters (Table 8). Parameters the reference car does not publish, such as losses, internal resistance and thermal masses, are estimates, each marked in the code and recorded in a decision record. When a reference test missed, only the estimates were tuned, never the official figures or the tolerances.

Table 8: **Main vehicle parameters.** Official figures of the reference car except where marked.

Motor peak power	230 kW	Test mass	2130 kg
Motor peak torque	360 N m	Drag coefficient	0.22
Reduction ratio	10.81	Frontal area (estimate)	2.35 m ²
Usable pack energy	82.5 kWh	Rolling resistance (estimate)	0.0110
Pack nominal voltage	550 V	AC / DC peak charge	11 / 150 kW

Table 9: **Reference tests.** Each is an automated test with a fixed $\pm 10\%$ window around the reference car's published figure.

Figure	Target	Window	Simulated
0–100 km/h	5.9 s	5.31–6.49 s	5.97 s
Range at a steady 100 km/h (100–0 % usable)	510 km	459–561 km	498 km
DC charge 10–80 %	37 min	33.3–40.7 min	34.0 min

Table 9 shows the three reference tests. Consumption at a steady 100 km/h is 165.6 Wh/km, and

180.1 Wh/km at 110 km/h, inside the 172–190 Wh/km band we used as the calibration anchor. The DC charge peaks at 150 kW. An AC charge over the same window takes 6.13 h, limited by the on-board charger.

7 Scenario results

Acceleration. Figure 10 shows a full-pedal start. Torque is flat at 360 N m up to base speed, after which power is held near its limit and torque falls. The run reaches 100 km/h in 5.97 s. Pack power exceeds shaft power by the motor and inverter losses.

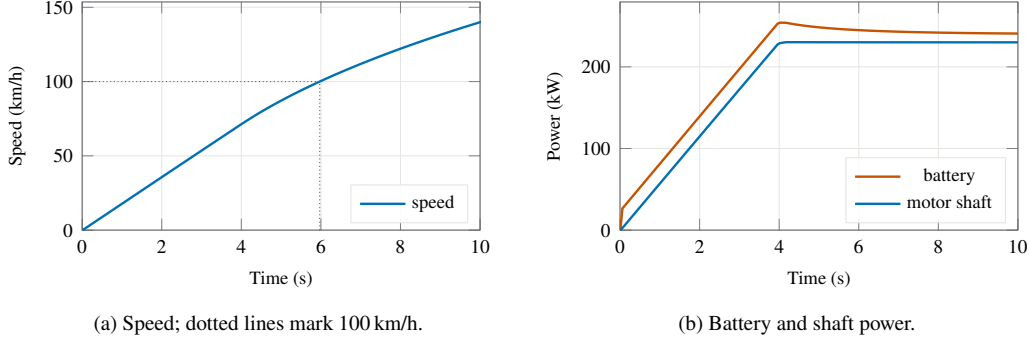


Figure 10: **Full-pedal acceleration from rest in Normal.**

Regenerative braking. From 100 km/h the driver lifts off for 4 s and then holds the brake at 30 % until the car stops, 8.0 s later (Figure 11). At 80 % SOC the BMS allows 60 kW of charge, so regen stays at that allowance (peak 56 kW at the pack) and the friction brakes supply the rest of the pedal's demand. Of the 824 kJ of kinetic energy at the start, 552 kJ (67 %) reaches the pack; the rest goes to road load, drivetrain losses and the friction brakes at low speed. The trip tracker shows 153 Wh recovered.

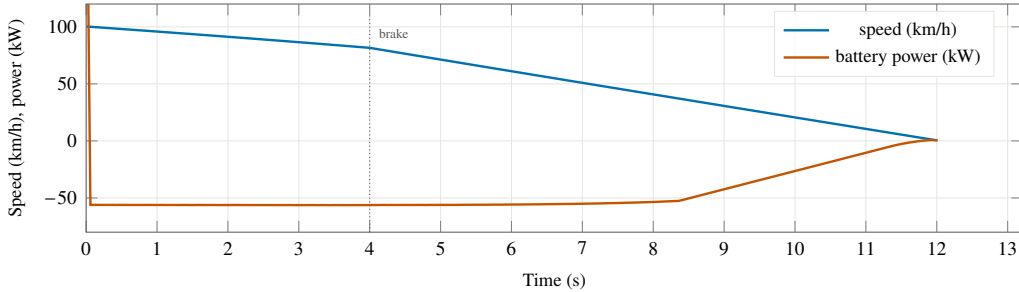
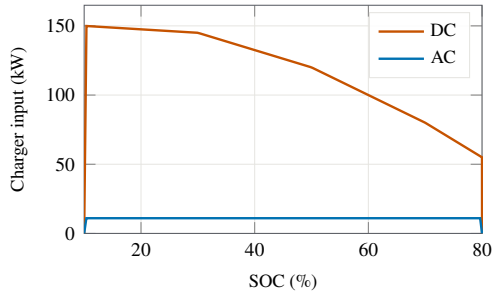


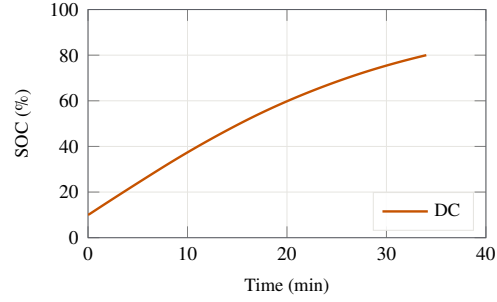
Figure 11: **Regenerative stop from 100 km/h.** Battery power turns negative on lift-off and is held at the BMS charge allowance through the braking, while the friction brakes take the remaining demand. Regen fades out below 5 m/s and the friction brakes finish the stop.

Charging. Figure 12 compares the two sources. The DC charge starts at the charger's peak and tapers as the pack fills; the AC charge is flat at the on-board charger's limit less conversion loss.

Drive cycles. A speed-tracking driver model follows the Low and Extra High phases of the WLTC Class 3 cycle [11] using only the pedal inputs (Figure 13). In Normal the Urban phase covers 3.10 km on 0.449 kWh and the Highway phase 8.26 km on 1.512 kWh. The modes differ little in energy (Table 3): on a fixed speed trace the wheel energy is the same, and Eco's stronger lift-off regen adds round-trip battery loss. The modes change how the car responds, not how much energy the trip needs. Over the Highway phase the motor warms to 39.3 °C while the pack, with its large heat capacity, stays at 23.1 °C.

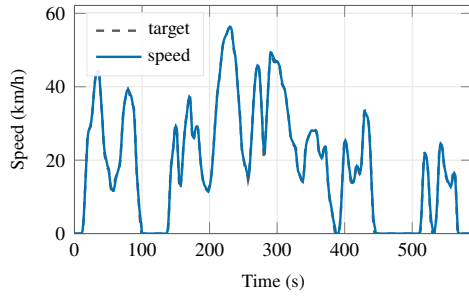


(a) Charging power against SOC.

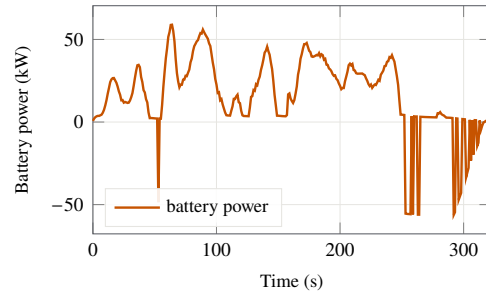


(b) DC charge, 10–80 % in 34.0 min.

Figure 12: AC and DC charging from 10 % to 80 % SOC. The AC charge takes 6.13 h.



(a) Urban (WLTC Low phase).



(b) Highway (WLTC Extra High phase).

Figure 13: **Drive cycles in Normal.** The driver tracks the target within ± 2 km/h for at least 98 % of samples. Negative battery power is regen; the spikes near the end of the Highway phase are the driver model pulsing the brake on the final deceleration, sampled once a second.

Fault response. Two identical full-pedal runs start from rest; in one, a cell over-temperature fault is injected at 2 s and restored at 5 s (Figure 14). The chain is visible on the bus. The BMS sends the DTC and a halved discharge allowance of 128 kW 60 ms after injection. The IC shows the warning after 80 ms and the VCU's drive decision names the cause after 160 ms. Battery power follows the allowance: at 4.5 s the faulted car draws 127 kW against 251 kW for the healthy one. Building this figure exposed a defect: the VCU had applied the discharge limit to shaft power, so losses pushed battery power about 8 % past the limit. It now applies the limit at the battery, and a test holds it there.

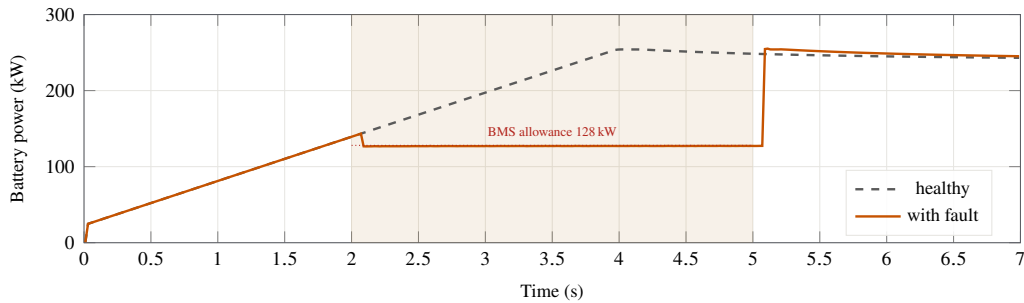


Figure 14: **Cell over-temperature while accelerating.** The shaded band is the fault. Power is capped at the BMS allowance and returns when the condition clears; the DTC stays stored until cleared.

OTA update. The update completes in 14.8 s from the first click (Table 5), of which the restart takes 1.8 s. The trace shows the TCU's progress, the power-down, and the new boot frame.

8 Verification

The project was built in phases from a written brief. Each phase had requirements, validation checks and small tickets, and each ticket had to pass typecheck, lint, the unit tests, the production build and the browser tests before it was committed. Decisions that the brief did not settle are recorded in twenty-one decision records.

- **Unit and simulation tests** (Vitest, 516 tests): bus timing, startup and its failure paths, gear interlocks, pedal maps and mode ramps, regen blending, AC and DC charging, faults and DTCs, thermal balance, drive cycles, telemetry, the OTA client and firmware banks, the guided demo scripts played headlessly, every view, and the reference tests of Table 9. The co-pilot's tools run against the real simulator, and its sessions, proactive triggers, report model and PDF against the scripted fake client.
- **Browser tests** (Playwright, 31 tests at 1366×768): every scenario unaided and again from the guided demo, the architecture trace, CSV export, an offline reload served by the service worker, a proactive suggestion accepted from the stage, and a PDF export. The browser tests run with a blank key, so they cover the co-pilot's unavailable state, not a live session.
- **Determinism**: identical inputs give identical snapshots and bus traces.

Of the brief's definition of done, the scenario, physics, offline and paper items are met and verified by the tests above. The public link is deferred by decision, and the build is ready for any static host. The review of every view against the design contract is a manual check. So are the live voice sessions: the co-pilot's usefulness by voice, in English and in Kiswahili, is measured by hand with a key, not by the automated suite.

9 Limitations

The model is a teaching and review tool, not a validated engineering simulation. The thermal model has one mass per component and two coolant loops. The battery has one resistance and no transient dynamics. The bus has no bit timing, arbitration or error frames. Many parameters are estimates, marked as such. The OTA verification step always passes; a real one would check a signature chain and roll back after a failed boot. The car drives on a rolling road with no steering or lateral dynamics. The co-pilot's replies come from a language model and are neither deterministic nor covered by the automated tests; its safety rests on the tool whitelist and the car's interlocks, not on the model following its instructions. Kiswahili speech quality depends on the live model, whose published language list we could not confirm includes Kiswahili. The API key sits in the local bundle, which is acceptable only because the build is not published.

10 Conclusion

Duma SDV turns an SDV software architecture into something a judge can operate. Six ECUs coordinate only through a catalogued bus, over physics that reproduces the reference car's acceleration, range and charge time within $\pm 10\%$. The fault, charging and update flows can be traced message by message. Because the core is deterministic and free of the user interface, the same code serves the live demo, the automated tests and every figure in this paper. Making the paper's figures from the simulator also exposed a real defect, the discharge limit applied at the shaft instead of the battery, which is now fixed and guarded by a test. The AI co-pilot follows the same rule as every ECU: it reaches the car only through a declared interface, so a language model can make the car easier to talk to without being able to override it.

References

- [1] Mehrdad Ehsani, Yimin Gao, Stefano Longo, and Kambiz M. Ebrahimi. *Modern Electric, Hybrid Electric, and Fuel Cell Vehicles*. CRC Press, Boca Raton, FL, 3rd edition, 2018.
- [2] Thomas D. Gillespie. *Fundamentals of Vehicle Dynamics*. Society of Automotive Engineers, Warrendale, PA, 1992.
- [3] Google. Gemini API documentation: Models, structured output and function calling. <https://ai.google.dev/gemini-api/docs>, 2026.
- [4] Google. Gemini live API: Real-time bidirectional audio with tool use. <https://ai.google.dev/gemini-api/docs/live-api>, 2026.
- [5] Michael Grieves and John Vickers. Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems. In Franz-Josef Kahlen, Shannon Flumerfelt, and Anabela Alves, editors, *Transdisciplinary Perspectives on Complex Systems*, pages 85–113. Springer, Cham, 2017.
- [6] International Organization for Standardization. ISO 15118-2: Road vehicles – vehicle to grid communication interface – part 2: Network and application protocol requirements. ISO, Geneva, 2014.
- [7] International Organization for Standardization. ISO 11898-1: Road vehicles – controller area network (CAN) – part 1: Data link layer and physical coding sublayer. ISO, Geneva, 2015.
- [8] International Organization for Standardization. ISO 26262: Road vehicles – functional safety. ISO, Geneva, 2018. Second edition.
- [9] International Organization for Standardization. ISO 14229-1: Road vehicles – unified diagnostic services (UDS) – part 1: Application layer. ISO, Geneva, 2020.
- [10] Ahmad A. Pesaran. Battery thermal models for hybrid vehicle simulations. *Journal of Power Sources*, 110(2):377–382, 2002.
- [11] United Nations Economic Commission for Europe. Global technical regulation no. 15: Worldwide harmonized light vehicles test procedure (WLTP). <https://unece.org/transport/standards/transport/vehicle-regulations-wp29/global-technical-regulations-gtrs>, 2014.
- [12] United Nations Economic Commission for Europe. UN regulation no. 156: Uniform provisions concerning the approval of vehicles with regards to software update and software update management system. <https://unece.org/transport/vehicle-regulations>, 2021.